



Texts of Plessey papers presented at the
International Switching Symposium
Massachusetts Institute of Technology
Cambridge, Mass., U.S.A. — June 6-9 1972

Papers:

Operating System of System 250	D.M. England	Plessey, Taplow Court
The Operational Requirements for Future Communications Control Processors	J.M. Cotton	Plessey, Taplow Court
Hardware of the System 250 for Communication Control	D. Halton	Plessey, Poole, Dorset
Telephone Switching based on System 250	W.A.C. Hemmings	Plessey, Liverpool

OPERATING SYSTEM OF SYSTEM 250

D.M. England

Plessey Company Ltd.
Taplow Court, Maidenhead,
Berks. England.

Abstract

The paper describes the Operating System of System 250 in terms of a series of abstractions achieving logical facilities convenient to the applications programmer and a unique Software interface between applications and operating systems code.

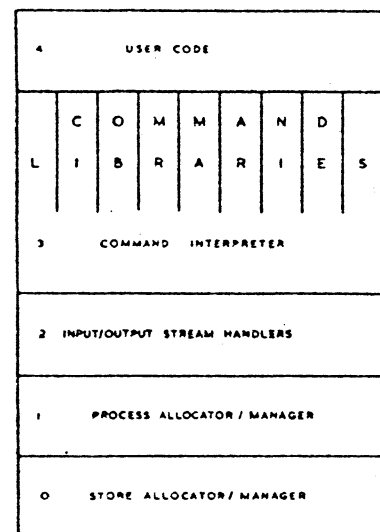
LEVELS AND ABSTRACTIONS

1. In a now classical paper on computer operating systems, Edsger W. Dijkstra postulates an operating system constructed as a series of layers or levels. Each level has two significant characteristics, the present paper being more concerned with the second:-

- (1) It offers a set of facilities to all higher levels in the operating system's hierarchy, enabling it to be built up layer by layer.
 - (2) It achieves some abstraction; some physical hardware entity (e.g. storage devices) is transformed into some smooth, logical, abstract entity (e.g. virtual memory) that is, moreover, altogether easier to manipulate.
2. The effect of Dijkstra's abstractions is as follows:-
- (1) At the bottom of the hierarchy rests the responsibility for scheduling processor time, allocating processing power to the competing programs able to run. Above this level, each program believes that it has a processor to itself; so long as it is furnished with the processing power it needs, the fact that it shares a processor with others can be disregarded.
 - (2) The "segment controller" draws a veil over the boundary between the main store and the disk. Above this level, a program simply asks for a "store segment" to be allocated. When the program tries to use this store segment, the segment controller ensures that it is available in main store. When an allocated store segment is not being used the segment controller transfers it onto the disk until it is next used.
 - (3) Programs higher than level 2 can converse with the operator via the console keyboard. Each program believes itself to be in sole communication with the operator and oblivious of simultaneous conversations.
 - (4) The software at level 3 is concerned with buffering input data streams and unbuffering output data streams, between the actual peripheral devices and the programs which process the data. Above this

level actual communication devices lose their identity, and programs communicate with the outside world through the medium of "logical communication units".

- (5) The independent sets of user programs can now reside in their private worlds, furnished in abstractions, whilst in reality they play a game of cox and box with the real resources of the processing system.
3. Threefold advantage is to be gained by this series of abstractions:-
- (1) The system is enabled to perform a variety of jobs simultaneously and without any interaction between them, making maximum use of the physical resources available.
 - (2) The user programs, which are after all the *raison d'être* of any processing system, are rendered independent of system hardware and thus robust to changes in the configuration and capacity of the equipment employed.
 - (3) Simplification of the user programs is achieved to the extent that they are concerned solely with the manipulation of convenient abstract entities whose behaviour is entirely logical and consistent.



1. BROAD STRUCTURE OF SYSTEM 250
OPERATING SYSTEM

AIM OF THE PAPER

4. The PP250 Operating System is constructed as a series of levels, a hierarchy of subroutines, in a manner similar to that suggested by Dijkstra. Fig. 1 depicts in broad terms the structure of the PP250 Operating System, each of its layers being discussed in subsequent paragraphs. This paper is more concerned, however, with the corresponding abstractions; to describe how these are achieved is to explain the principles upon which the system is based. Thus, given the hardware mechanisms described in companion papers, the aim of this paper is to show how the physical resources of the system, processors, storage media, and peripheral devices, are transformed into abstract resources, processes, virtual memory, and data streams.

PROCESSORS TO PROCESSES

5. To show how CPUs suffer loss of identity it is necessary to anticipate the discussion on virtual memory to the extent of assuming a system of dynamic storage allocation. This implies the existence of privileged code, in the dungeons of the operating system, with the power to create on demand blocks of the requisite size and deliver capabilities for them.

6. A program may be thought of as a static structure of code blocks and constant data blocks contained in a network of constant capability blocks. It must have a particular start point, a particular code block within this network. Apply a CPU at this start point and it executes code, it processes input data, it asks for data blocks and is delivered capabilities, and so it has to ask for capability blocks in which to store them. In short, it constructs its own private sub-network of capabilities, a private data structure unique to this run of program.

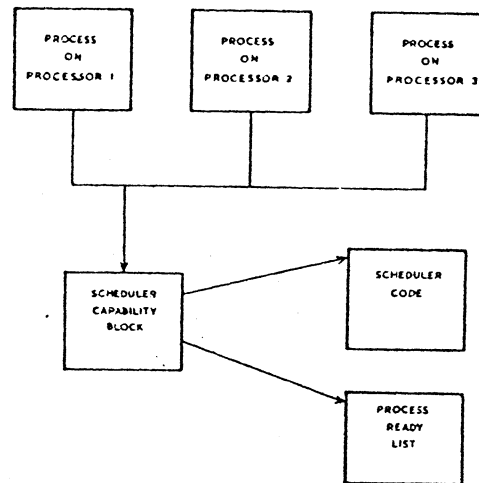
7. Apply another CPU at the same start point, perhaps at the same time, and although it obeys the same code it asks for its own blocks and constructs a separate data structure. Allow discontinuities in each execution of the program, e.g. while waiting for more input data, and the two runs can be performed on the same CPU, whose power is shared or scheduled between the two runs.

8. Two conclusions can now be reached:-

- (1) It is possible to distinguish between execution of a program and the program itself. Although there may be many more or less simultaneous executions of the same program, the program itself is singular. The term used for the execution of a program is a "process" and that for the kind of program which can have many simultaneous executions is a "reentrant program". In a telephone switching system, for example, there may be many processes, each servicing a different telephone call, but all obeying the same reentrant telephone switching algorithm.
- (2) It is immaterial to a set of processes whether each has its own CPU or all are scheduled on the same CPU, so long as each receives a requisite amount of processing power.

9. As a system normally contains a number of different programs, so it is possible to have processes of different types simultaneously in existence.

10. CPUs have now almost disappeared, short of explaining the mechanism for scheduling processes in a multi-processor environment (see Fig. 2). The action of this mechanism is to select another process to run on a CPU when the current process on that CPU comes to an end or has to await some event, e.g. arrival of more input data:-



2. REENTRANT PROCESS SCHEDULER

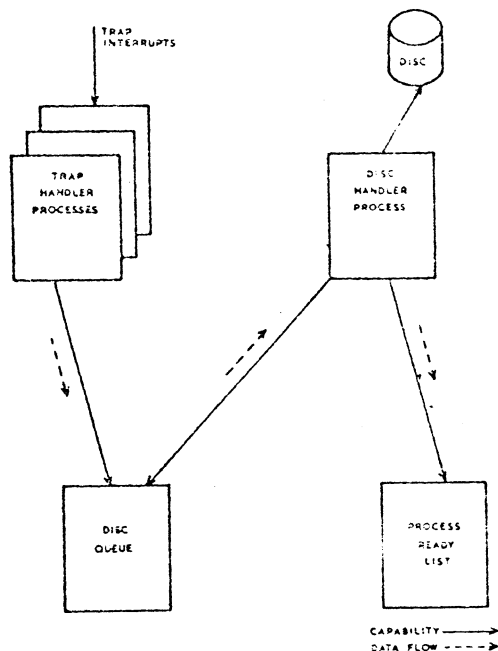
- (1) It is part of the action of every process which runs on a CPU select the next process to run on that CPU by executing a "process scheduler" subroutine.
- (2) The process scheduler selects the first item in a "process ready list" of processes that are in an executable condition.
- (3) As there is only one process ready list for the entire system, it provides a common work pool for all CPUs. A process might run on any CPU, whichever selects it when it reaches the head of the process ready list. Further, where there are discontinuities in the running of a process, different parts of its run might be performed on different CPUs.

11. It may now be seen that CPUs do no more than furnish processing power to processes, the number of CPUs in any configuration being dictated solely by its total processing requirement, the processes themselves being insensitive to the number of CPUs present and the actual CPUs on which they run. Like Dijkstra's processor allocator, the process scheduler furnishes to each process an abstract CPU which meets its total processing requirement. The abstraction of CPUs is thus achieved.

VIRTUAL MEMORY - STORAGE MEDIA TO BLOCKS

12. The purpose of this particular abstraction is to eliminate the distinction between main storage and backing storage and enable a process to be concerned solely with store blocks of the requisite size and type of access. The subject is a large one and can only be given cursory treatment in a paper at this level.

13. The aim of the virtual memory scheme is for blocks to be transferred between main store and disk automatically, without the processes using these blocks being conscious that any such transfers are taking place. The only requirement that a process has is that when it attempts to use a block, the block should automatically be transferred into main store if it is not there already. A simple hardware mechanism is used to detect an attempt to use a block which is not currently in main store. This causes an interrupt into a "trap handler" process (fig. 3) of which there is one per CPU.



3. TRAP AND DISC HANDLERS

14. The action of the trap handler process is to queue the trapped process, with an indication of the required block, for action by the "disk handler" process. The disk handler initiates transfer of the required block into main storage space allocated for the purpose. It then enters the trapped process in the process ready list. When the process is rescheduled it successfully reattempts the trapped instruction, and proceeds to use the block concerned as if nothing had happened.

15. When a process requires a new block of storage, it calls a "store allocator" subroutine, specifying the length and access required. The store allocator allocates appropriate space on the disk. It delivers to the process a capability for the block thus created. An attempt to use the block would, according to the mechanism described above, result in the block being transferred from disk into main storage space allocated for it. In fact, the actual transfer is in this case short-circuited, as the block does not yet contain any useful information.

16. When a process has no further need for a block, it overwrites all its capabilities for the block. The system may reuse disk space occupied by a block when no capabilities for it exist, as the block is then inaccessible to all processes.

17. In this summary of the virtual memory scheme, it remains to outline the algorithm used to select blocks for transferring out:-

- (1) Essentially, the algorithm employed is "longest in", it selects for transferring out the block which has been longest in main store.
- (2) However, the algorithm permits certain critical blocks to be locked down in main store, e.g. those used by the disk handler.
- (3) When a block can be recognised as having remained unchanged while in main store, i.e. if access to it is limited to read data, and/or execute, or read capability, the space it occupies is deallocated without any actual transfer out having to take place.

18. Although the mechanisms required to achieve a virtual memory are quite involved, its use is simplicity itself; a process simply executes the store allocator when it requires more store and overwrites all copies of capabilities for blocks it no longer requires. This discussion makes no distinction between data type and capability type blocks. It makes the tacit assumption, therefore, that capability blocks can be held on disk, indeed that the capability network extends from the address space of main store to the address space of the disk, regardless of the physical boundary between them. It is essential for this to be so for a virtual memory, the abstraction of storage devices, to be achieved.

DEVICES TO DATA STREAMS

19. The purpose here is the abstraction of non-interactive devices, paper tape readers, paper tape punches, lineprinters, etc. in favour of "data streams". A process may then be concerned simply with its input and output data regardless of the actual devices employed.

20. When a process requires to input or output, it must first execute a "stream allocator" subroutine, supplying the stream name, in keyboard characters, as a parameter. Correspondingly, a human may use a keyboard command to ascribe that name to a device. The action of the stream allocator is to allocate to the process the device thus named. The process may then either input data from the stream or output data to it by executing a "get data" or "put data" subroutine respectively, some suitable unit of data e.g. 100 words, being thus transferred.

21. The process is insensitive to the device, even the type of device, used to transfer its data. The "put data" or "get data" subroutine simply passes a unit of data to or from a device handler process associated with the device concerned and responsible for the physical transfer of data.

22. The human may ascribe a name to a data file within the virtual memory instead of a physical device. The action of the stream allocator is then to allocate the data file to the process as its data stream. Physical transfer between the data file and a device is then wholly disassociated from the process, to achieve a conventional "offlining" facility. The process, however, still employs "get data" and "put data" as discussed above to communicate with its streams. Thus, a process is insensitive not only to the devices providing the source and destination of its data but also to whether or not that data is off-lined. Data streams correspond to Dijkstra's "logical communication units".

FORM OF A LOGICAL RESOURCE

23. To be complete, this paper must answer the question "What, then, is a logical resource?" In the case of a store block the answer is self evident; the store block is itself the resource. A more complex resource consists of a collection of store blocks which can contain, in some meaningful format, data describing the resource. To take an example, the format for a process must provide for its start point, priority, space to preserve register values on interrupt or suspension, etc. A logical resource, then, is nothing more than a data structure, of which a single block is the simplest example.

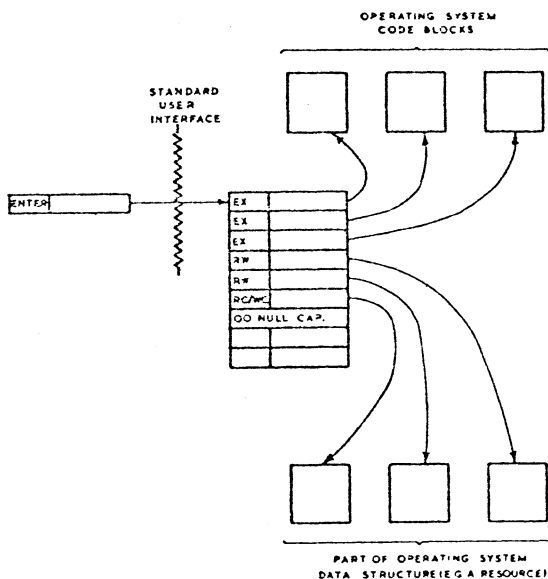
24. The paper now goes on to show that, just like a store block, any logical resource is represented to the user by a single capability. The paper describes how the operating system is called and how abstract resources are represented, allocated, and operated upon. It shows the realisation of a standard software interface.

CALLING THE OPERATING SYSTEM

25. Calls to the operating system are invariably achieved by means of an "enter type" capability, as described in companion papers. This means that, in order to call the operating system, each process has to be furnished with an enter type capability for operating system subroutines (fig. 4). This enables the process to execute code blocks within the operating system, which thus carries out the required operations. Two points should be observed:-

- (1) The enter mechanism automatically denies to the calling (user) subroutine access to the called (operating system) subroutine's data structure, thus protecting the operating system from corruption by user processes.
- (2) Thus, there is no formality involved in a call to an operating system subroutine, which is called just like any other. Unlike conventional systems, no special mechanism is required to enter the operating system, which in turn requires no "privileged mode" of operation.

26. Each process is furnished with an enter type capability for a "resource allocator" within the operating system. This gives the process access to a number of code blocks, each one being concerned with the allocation of a particular resource-type, the "store allocator", the "stream allocator", etc. execution of which causes a logical resource of the corresponding type to be created and allocated to the process, as described below.



4. USER INTERFACE/ACCESS TO A RESOURCE

RESOURCE REPRESENTATION AND ALLOCATION

27. Execution of the store allocator causes a store block of the required size to be created and a capability with the required access to be delivered. This type of resource might be thought of as a special case; once allocated and represented by a capability, a store block can be operated upon directly by means of CPU instructions, whereas all other resource-types can only be operated upon by calling the operating system. The abstract machine, however, is insensitive to such distinctions; it is no more than a convenience of implementation that operations which are performed on simple data structures, individual

blocks, are achieved by hardware, whereas those on more complex data structures are achieved by software. An abstract resource is no more than a particular data structure of blocks, the action of its resource-type allocator being as follows:-

- (1) It creates an appropriate resource (data structure) and delivers a capability for it. This is a direct parallel with the store allocator.
- (2) In this case, however, the capability delivered is an enter type capability enabling suitable code blocks to be executed to perform operations on the resource (data structure).
- (3) Here is the curious and intriguing point, that this is an interface to the operating system specially created by the resource-type allocator to perform operations on the particular resource concerned. The resource-type allocator obtains a capability block and inserts into it execute type capabilities for the standard code blocks used to perform these operations and for the constituent blocks of the resource's data structure. Fig. 4 still applies.

A STANDARD, DYNAMIC, AND ADAPTIVE INTERFACE

28. It should be observed that, when a user process employs its enter type capability to execute a resource-type allocator, it is (excepting storage) delivered another enter type capability with which to perform operations on the resource thus allocated. The two invariable rules constituting a standard user interface with the abstract machine can now be stated formally:-

- (1) A logical resource is always represented by a capability, which can be used to perform permissible operations on that resource.
- (2) A call of the operating system is always achieved by means of an enter type capability.

29. Although standard, the user interface can now be seen as completely dynamic. When a user process requests a resource, it is furnished with its own private interface to the operating system, which it can use to perform operations on that resource. Further, this dynamic interface adapts to a user's requirements; as his processes request resources of various types, the delivery of corresponding capabilities extends the scope of his user interface to provide just those operating system facilities he requires.

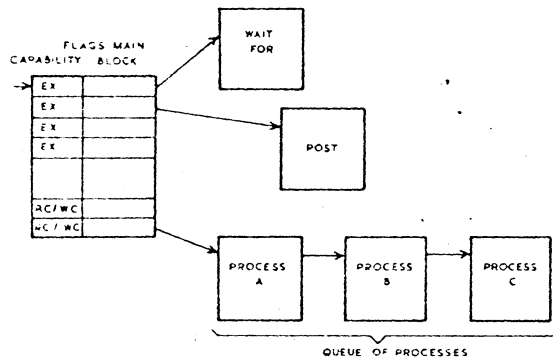
SYNCHRONISING FLAGS

30. A further resource (data structure) is normally employed for synchronising processes and communicating between them. This resource is called a "flag", and may be created by executing a "flag allocator". The problem which the flag mechanism is designed to overcome is that, while processes perform asynchronously, they nevertheless require to synchronise the transfer of information between them. The concept of flags is best understood in terms of the operations that can be performed on them:-

- (1) When a process has a message to send to another process, it "posts" the message "on a flag" for which both have a capability, it executes a block of code that records the message within the flag's data structure.
- (2) The receiving process is delivered the message on request, it executes a block of code that extracts the message from the flag's data structure. However if the receiving process requests the message prior to its being posted, it is suspended until the sending process has performed the post operation. The receiving process is said to "wait for" a message "on a flag".

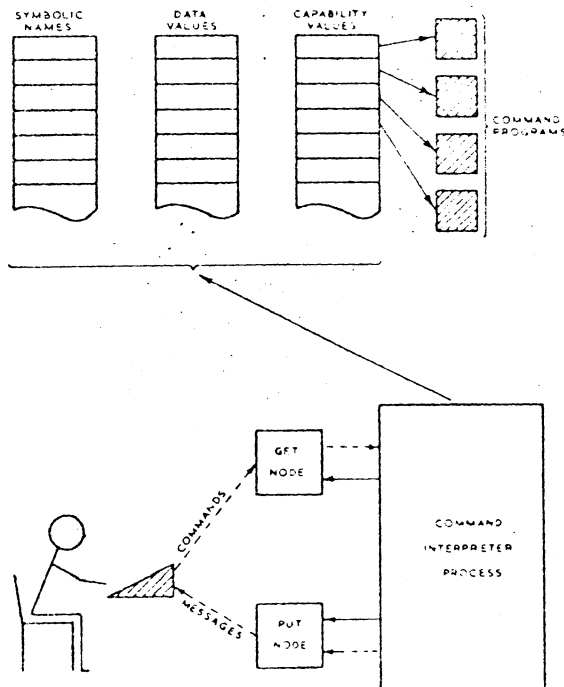
31. Thus, the operations that can be performed on a flag are to post a message and to wait for a message. Although the processes

involved perform asynchronously, the flag mechanism imposes synchronisation on the transfer of messages between them by ensuring that posting and delivery occur in the logically correct sequence. The flag concept permits a number of processes to wait for (fig. 5) or post messages on the same flag. Such messages would be waited for in turn and delivered in posting order. Also, a process may wait for a message on more than one flag, where it can expect one of a number of possible events, e.g. in a telephone switching process, a message indicating the next dialled digit, the return of the handset to its rest, or a timeout.



5. BROAD DATA STRUCTURE OF FLAG THE HUMAN INTERFACE

32. When a human approaches a terminal and operates the "attention key", this causes a "command interpreter" process to be created, the purpose of which is to interpret and obey each command subsequently typed at the terminal keyboard (fig. 6). As the only action the system is able to take in obeying



6. SYMBOL TABLE

such commands is to allocate and manipulate the resources of the abstract machine, a command must be compounded from a basic set of commands, each of which correspond to an abstract machine facility. This basic command set extends the abstract machine to the terminal, where its facilities are then available for the human to create and manipulate abstract resources at will.

33. When created, a resource is represented within the system by a capability. When the human creates a resource at a terminal he gives it a symbolic name. These are respectively the internal and external representation of the resource; each reference that the human makes to the resource is replaced within the system by its associated capability. Such associations are recorded within a "symbol table", which is part of the data structure of the command interpreter process.

34. Each command name is also recorded in the symbol table, its associated capability defining a "command program" which is executed by the command interpreter in order to obey the command. Whenever the human types the command, to create or manipulate an abstract resource, it is this command program which calls the associated abstract machine facility on his behalf.

35. The human is permitted to generate new commands, with their own symbolic names, being compounded of existing commands. This enables him to manipulate a number of resources with a single command, which may then be expressed solely in terms of the work he requires the system to undertake and independently of the abstract resources actually involved.

THE USER

36. To conclude this paper, the main concerns of a system user, in expediting the design of his application, may be summarised as follows:-

- (1) The standard software interface is as pertinent to applications programs as it is to the operating system. It is a task of the user, therefore, to identify logical resources (data structures) convenient to his application, e.g. a telephone subscriber, and the operations to be performed on those resources, e.g. send ring tone. He may then proceed to write corresponding resource-type allocators and the code blocks which achieve the required operations.
- (2) The user must determine the types of processes required to achieve his application and the criteria by which such processes are created, e.g. a telephone switching process might be created when it is detected that a subscriber has raised his hand-set.
- (3) There may be a range of applications personnel who are required to communicate with the system for different purposes, e.g. operational staff, maintenance staff. The user must determine the terminal facilities required by such personnel and establish corresponding command structures. He may then proceed to define the command programs which achieve these facilities, using for this the command generation commands.

Acknowledgement

I would like to thank the many colleagues on whose work this paper is based and the Directors of the Plessey Company for permission to publish it.

THE OPERATIONAL REQUIREMENTS FOR FUTURE COMMUNICATIONS CONTROL PROCESSORS

J.M. Cotton

Plessey Company Ltd.
Taplow Court, Maidenhead,
Berks, England.

ABSTRACT

The future communications control problem will encompass not only the control of individual systems but also the control of remote switching systems, over data links, from a central location. In view of the evolutionary nature of Telecommunication networks, it is necessary to provide for both local and remote control with the same equipment. Additionally, it should be possible to control simultaneously, or with the same type of equipment, message switching systems, and other real time control applications.

These requirements along with considerations of software reliability point to the necessity of preventing undesirable interaction between the various tasks of what is, in effect, a complex real time system.

This paper presents the operational requirements of such a control system, and deduces the necessary system features. Accompanying papers describe the software structures implicit in this approach and the hardware designs both for the control equipment and for the compatible new telephony system.

1. GENERAL

Before considering the requirements for the control system, based on Stored Program Processors, suitable for future communications networks, it is important to be clear what potential advantages might be gained from the use of such a control system. These may be listed as follows:

- a) The control functions in an exchange or network would be entirely separate from the switching, signalling and transmission functions and each could develop separately without impact on the other.
- b) Ideally the logic functions normally distributed throughout switching equipment, relay sets, markers etc. would be replaced by computer instructions executed in accordance with a program. A change of facility, demanding a rearrangement and redesign of logic and hardware in a conventional exchange which makes retrospective modification expensive could be provided by program change or replacement in an SPC system.
- c) Economies could be made in the design of the switching system, since control would have an overall view of the traffic pattern and could organise connections on a dynamic basis.
- d) Very fast call set-up times could be achieved, limited only by the operating response of the switching elements. As a result an SPC network could have better revenue earning characteristics.

- e) There need be no significant differences between exchanges at different levels in a hierarchy except size and traffic handling capacity.
- f) Inter exchange communications could be organised via data channels thus removing the need for expensive signalling relay sets, and freeing the speech paths from any constraint imposed by in-band signalling.
- g) Statistical information relating to system performance, network limitations etc., would be readily available as a by-product of Processor Control.
- h) Reorganisation of routing strategies could be permitted to avoid traffic restrictions due to cable breakdown, switch block blocking etc. This reorganisation might be automatically executed in program or might be initiated by some manual input from a network management centre.
- i) The control equipment should require less maintenance and would be able to provide more comprehensive diagnostic assistance for such maintenance as is required.
- j) New switching technologies which may be developed in the future should be more easily accommodated when the control function is separated and provided by computer programs.

2. ADMINISTRATION POLICIES

It is well recognised that for any communications system the complete failure of the control system is catastrophic; hence it is undesirable for the control to have a probability of failure higher than once in 50 years or more depending on the number of subscribers served. In order to achieve this degree of reliability with present day components it is necessary to organise the control processing system with adequate redundancy and facilities for control reconfiguration. In consequence the minimum viable control system is relatively expensive, and there is a minimum number of subscribers which may be economically served which is variously quoted as between 2000 and 5000 telephone lines. Such an installation should be capable of handling 10,000 lines with no further control equipment. Hence, the approach which may be taken by an administration to the installation of "Stored Program Control" (SPC) systems will depend on the existing network circumstances and its financial policy.

If the nation concerned is relatively undeveloped with little existing communication equipment, then the only hindrance to its being able to take full advantage of the flexibility and evolutionary possibilities of SPC may be doubts about the economics of the initial installation. Such doubts might well be removed if the equipment were capable of controlling several exchanges in an area, or a number of different communication

services simultaneously (e.g. telephony and message switching) or if the administration could make use of the spare processing capacity for data processing purposes of varying kinds. If, however, the nation already has large quantities of existing electro-mechanical equipment installed at various stages in its useful life, then its problems are more severe. The three obvious policies which it may take can be summarised as renewal/extension, replacement and overlay. These are discussed below.

- a) Renewal/Extension: If the Administration follows a policy of only renewing with SPC those installations which have reached the end of their life and providing SPC extensions where they are of sufficient size to be economic, then many decades will pass before SPC makes any significant penetration in the network and the facility and service benefits become apparent. This will be particularly true of some of the benefits (d, f and h above) which only become apparent when there is a high penetration.
- b) Replacement: Under this policy exchanges which require extension are completely replaced by the new system. The size of the new installation will thus usually be big enough for the SPC to be economic. The drawback is that large capital investment is required.
- c) Overlay: Under this policy the administration will plan to incorporate the modern equipment in an "overlay" network, separate from but interfacing with the old equipment at suitable points and to interconnect the overlay portions of exchanges in a network divorced from the existing network, and only connected to it at strategic points. An exchange growth treated in this manner, may continue to expand to meet growth at the requisite planning periods until it serves the major portion of the telephone service area concerned. At some stage when economic, the old equipment is removed and the total exchange load is now handled by the modern equipment.

It is inevitable that in the overlay principle with the modern equipment being provided to satisfy growth, it will be provisioned in smaller sizes than would be the case with complete exchange replacements, e.g. suppose it is required to extend an 8,000 line telephone exchange with a projected annual growth of 5% and the planning period is 5 years. Then the modern equipment to be installed will provide for an ultimate of 2,200 lines at the 5 year date. This may well not be economic, and smaller installations would certainly not be economic because of the cost of the individual control systems.

An alternative approach is to aggregate the growth of a number of exchanges within an area, say a G.S.C. area and control the equipment provided to switch growth traffic at each of the individual exchanges from a common central processor installation. Then if there are say 10 exchanges within the area with an average 5 year growth of 1,000 lines per exchange, the SPC installation is now economic - there are of course some additional charges to be borne in this case for the provision of data links and associated equipment.

3. NETWORK BASED ON SPC CONTROL CENTRES

It has been shown that to apply SPC to the growth elements of an existing network using an overlay concept is only likely to be economic if a control centre is provided, having facilities for controlling the growth equipment of a number of exchanges in the area. Sufficient work has been

done to indicate that even if an Administration adopted a policy of replacing existing exchanges with SPC, the technique of controlling the area from a central processor would still be the preferred choice. The basic reasons for preferring area control are:-

- (i) It ensures that the control complex is worked at a reasonable level of efficiency.
- (ii) Because of centralised control it is possible to "map" the switch-blocks and interexchange connection circuits for a whole area. In the case of the switchblocks the use of a central map permits switch-block design to be optimised in terms of cost.
- (iii) The ability of a processor to select an "overall path" for a within area call improves network efficiency since a call from a subscriber on exchange A to one on exchange B (either direct or via exchange C) is only set up if links are available and the 'B' subscriber's line is free. Thus the revenue earning power of a given network is improved.
- (iv) The location of processors in a central situation is more conducive to efficient maintenance. The central processor armed with efficient diagnostic programs can identify faults perhaps down to plug-in card level and provide a print-out of the fault and diagnosis. From the centre a maintenance officer can be despatched to the fault location equipped with the necessary replacement unit.
- (v) Management of a network is more readily achieved in terms of alternative routing strategies designed to "route around" fault equipment or junctions.
- (vi) Day to day administration of a network e.g. disconnection of service for ceased lines or non-payment of rental, changes in class of service etc. can be achieved from central control.
- (vii) The gathering of management statistics on traffic, call holding time, call distribution by destination, levels of successful calls etc. are readily available with the appropriate programs.
- (viii) Inter-processor communication is only required on those calls entering or leaving the area.
- (ix) The improved facilities and service offered by SPC achieve a wider and faster penetration.
- (x) P.A.X. and P.A.B.X. equipments within the area may also be controlled remotely by the central processor equipment.

4.

SYSTEM CONCEPT

Ideally to meet the requirements of an evolving network either of an overlay nature or by exchange replacement, the new telephony system should possess two important properties. These are:

Flexibility

Ability to evolve

The requirement for flexibility arises from:-

- (i) variations in size for different environments in terms of subscribers and traffic.
- (ii) variations in facility requirements in different environments.
- (iii) the need for the system to exist in many varying configurations i.e. main exchanges; group switching centres; overlay network switching nodes etc.

The requirement for the ability to evolve arises from:-

- (i) qualitative changes in the network due to the introduction of new services and facilities.
- (ii) the rapid advances in technology.

From these requirements the following general principles can be defined which could lead to an attractive system framework:-

- a) General purpose approach to the provision of equipment; which allows both flexibility and evolution.
- b) Open-endedness of organisation; allowing unrestricted growth.
- c) Recognition of natural distinctions within the system such that the distinct parts can develop separately with the advent of new needs and means.
- d) Modularity, both in hardware and software, which will provide flexibility of application.

These principles will be followed by division of the total system into sub-systems which are functional entities rather than physical entities. A sub-system is thus implemented as a modular combination of hardware and software which is functionally complete such that any change to the sub-system is contained within itself and should not necessitate associated changes in other subsystems.

A sub-system is defined by means of its functional interfaces with the rest of the system. The definition thus includes both the hardware and software interfaces which envelop the sub-system. Complete freedom of design and evolution and varying hardware/software boundaries are possible within the sub-system interface envelope. The rest of this paper is concerned with the control sub-system, which, in the network control situation might be thought of as a processor utility since it could be used to control telephony sub-systems of successive equipment generation and designed by different manufacturers, as well as other communications sub-systems such as data networks, telex and message switching. The telephony sub-systems are discussed in a companion paper.

5. SYSTEM CONSTRAINTS

The constraints which are imposed on the design of a processor complex for it to be suitable as the exchange control or network Control Centre (CC) in the future communications network as described above, may be listed and discussed as follows:-

- (i) Continuity of Service is required for all the exchanges in the network with no service interruption allowed during expansion of hardware or facilities. This is even more important if a Control Centre is supplying the control for all the exchanges in an area. Continuity of service is also required despite the failure of portions of the control system so that on a probabilistic basis its mean time between failures must be between 40-325 years (dependant on amount of equipment controlled) in the face of both transient or permanent hardware failure and the inevitable existence of software errors. Such service continuity cannot rely on manual intervention since the control equipment may spend long periods unattended.
- (ii) Expansion of the number of subscribers in an area is presently at 6% p.a. in the U.K.

If an overlay technique were followed then the increased traffic only would be taken by the Control Centre. Assume the initial CC installation is economically justified in an exchange or an area to cover the first five year expansion, then the next 25 years will bring a 13 : 1 expansion in the number of subscribers served by the CC which will represent an even greater increase in traffic. If during this period the original outdated electro-mechanical equipment is also replaced then the CC traffic will be increased by a factor greater than 16 : 1. It is unlikely that an economic initial CC installation will be able to expand its load by more than 4 : 1 without increase in the number or power of the processors. Thus the CC must be capable of at least 3 : 1 expansion in power during its life, and such an increase in processor power should not necessitate reprogramming of the system. For area control the CC must also cope with evolution of both the network and the CC itself during this period as a result of the introduction of more advanced technologies, and evolution and increase in number of the facilities to be offered to the subscriber.

(iii) Network Control by the CC requires that some of the very large number of low activity peripherals which form the exchange equipment will be controlled remotely over data links at the same time as the CC is in direct control of co-located equipment. The network will (in the UK at least) consist of telephony sub-systems designed and programmed by different manufacturers, as well as the administration itself. The CC hardware and associated operating system must therefore be able to support many different interacting software sub-systems written by different teams at different times throughout its life and with varying degrees of competence. Also it will probably be required in the future to support simultaneously other tasks which must not interact with the telephony task.

6. REQUIREMENTS

From the foregoing discussions it is possible to extract the essential operational requirements which will influence the design of a suitable processor complex for a Control Centre for either network or individual exchange control. These may be considered under the following six headings:- modularity, continuity, expansion, evolution, isolation and automatic recovery.

- (i) Modularity of the equipment is necessary to allow economic provision of redundancy for system reliability, and expansion as nearly graduated to enhanced requirements as is economic.
- (ii) Service Continuity of the P.U. must be maintained for the life of the system with a statistically probable mean time to system failure from all causes of 50 years or longer. The continuity must be maintained despite the addition, subtraction, modification or evolution or maintenance of both hardware and software modules. Acceptable degrees of degradation as a result of partial system failure must be specified by the administration.

(iii) Expansion of the Control Centre equipment has been shown to be necessary by a factor of 3 or more during its life time. This brings an important requirement for an adequate addressing range for the maximum size of control area envisaged for both peripherals and storage.

The second requirement is that addition of storage or processing power should not demand any alteration of programs.

(iv) Evolution of both the hardware and software of the CC must be expected during the life time of the system, which hopefully will be 25 years upwards. The hardware will need to evolve to take advantage of improved technology and provide new or improved resources: whilst the software will evolve to provide new application facilities, and new operating system facilities and control of new resources. The need for software evolution points to the requirement for system features to allow subsequent on-line modification of the operating system and application programs. Such modification will almost certainly be under-taken by other than the original design staff.

(v) Isolation of the various software sub-systems is a requirement to provide adequate insurance that:-

- a) Sub-systems (or users) are prevented from corrupting or illegally accessing the resources of other sub-systems.
- b) Sub-systems are prevented from corrupting or illegally accessing the resources of the Control Centre.

The isolation is particularly important in the area control case where sub-systems of differing purpose, design generation and manufacturer must be co-ordinated. Such isolation must, however, be controllable to allow necessary and defined interaction between the various sub-systems and between each sub-system and the CC operating system.

(vi) Automatic Recovery from the effects of any type of hardware or software malfunction must be designed into the system. Software is vulnerable to corruption by hardware or software failures. Hence, the software faults and hardware failures should be detected, not necessarily as soon as they occur, but before they can corrupt any other sub-system. Mechanisms must be provided to automatically isolate a suspected fault sub-system from the system until it is no longer considered suspect or has been repaired.

7. DESIGN OBJECTIVES

The general requirements, discussed so far, for the control system either for a single exchange or for an area Control Centre have been interpreted into the design objectives of the System 250. These are discussed below:-

- a) Since the requirements for increased processing power and store in general do not occur simultaneously, it has been an objective to have these separately expandable in what is known as a

multi-processor organisation. The most economic form of redundancy is $n + m$ where n is the number of modules for the task and m is the spare spares to allow automatic recovery in the face of module failure. It is desirable that $n > m \geq 1$. It is also desirable that the modules be small since the M.T.B.F. of a module is proportional to its size or complexity and its M.T.T.R. is inversely proportional to its size - both effects improving the mean time to system failure for any given value of equipment.

- b) The requirement that addition of storage or processing power should not demand any alteration of programs, reinforced by the $n + m$ redundancy philosophy leads to the objective that the processors should not be dedicated to any particular task but should be capable of being regarded as part of the system resources. This brings the important corollaries that control of the following activities must be dedicated to any particular processor but must be considered as system functions.

System timing
scheduling
I/O handling
interrupts

It is also important that all code should be re-entrant so that so that processing power may truly be regarded as a resource allocatable as the situation demands.

- c) To facilitate hardware evolution it is an important objective that the various interfaces between hardware modules should be as few and as simple as possible.
- d) To facilitate software evolution and to improve the ability of the system to provide service continuity despite the inevitable software bugs it has been an objective to take software isolation down to a much finer structure than the sub-system level. Each code block or sub-routine must have isolation against interference from other code blocks.
- e) Such isolation together with the requirements for expansion without software alteration, and reconfiguration in the face of store faults leads to the objective of flexible and fully relocatable storage partitioning and allocation. It would also be most desirable to provide the level of software abstraction which allows each sub-system to believe it is the only occupant of the system.
- f) As a corollary of objective b) it has been an important objective to provide a high degree of fault detection in hardware so that each processor in the system will detect its own faults. Having detected a fault it should have hardware facilities for removing itself from the system and applying self-checking until either it finds itself fault free (in the case of transients, software faults or store faults) or it is repaired.
- g) Since the mean time to system failure is inversely dependent on the time to repair it has been an objective to provide facilities for a good processor to be used to diagnose faults in another processor.

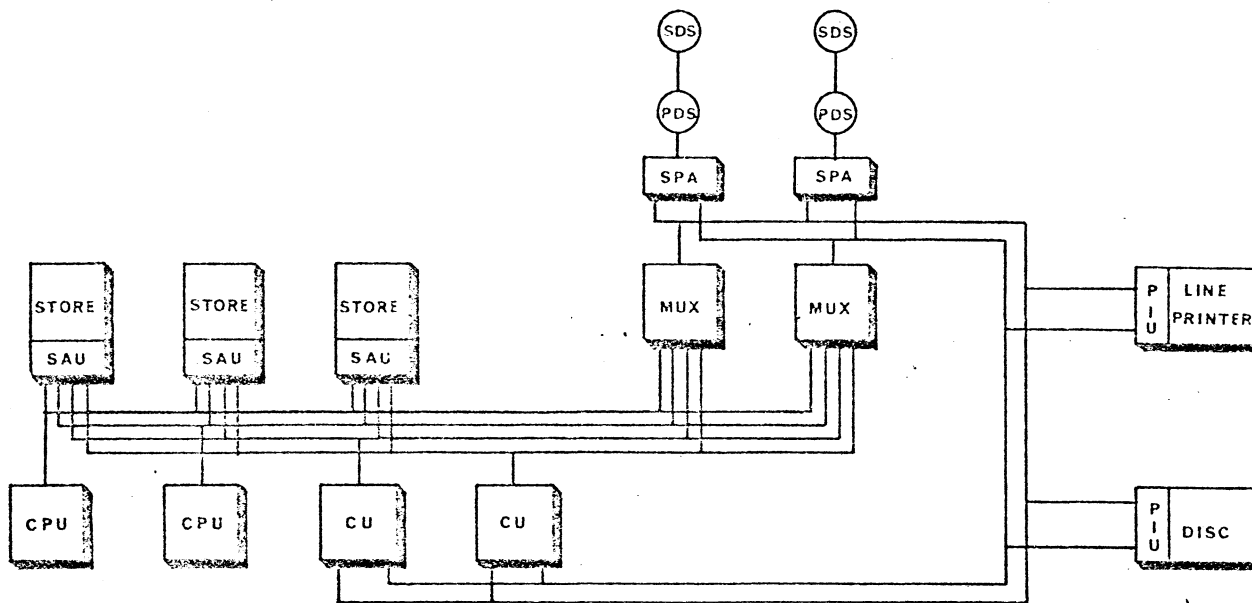


FIG. 1 TYPICAL SYSTEM CONFIGURATION

8. SYSTEM REALISATION

The System 250, which is described in detail in accompanying papers, is the system which realises all the objectives and meets the requirements discussed above. It is fully modular with simple interfaces as will be appreciated from the typical system organisation shown in fig. 1. It is possible to add storage or processing power independently without need for software alteration, and the additions may be made to an operational system without disturbance to service.

The key to the ability to isolate software modules and detect both hardware and software errors whilst preventing their effects from corrupting the remainder of the system has been the use of Capabilities. The original concept came from Van Horn (ref. 1) but in System 250 they have been realised in hardware. Fundamentally a capability defines a system resource, the most easily understood examples being a store block or a subroutine.

Fig. 2 shows that the position of the store block is defined by base and limit values whilst the permitted use of the block is defined by the access field. Such a capability may be the exclusive property of a process but if desired the owning process may allow other processes to have the same or more restricted access to the resource defined by the capability.

By means of such capabilities it is possible to run many entirely independent jobs on the same system, with complete control of the isolation or desired interaction of the jobs.

By these means it is possible to test undebugged software on a live system without fear of system collapse. Thus it becomes feasible to use the same control system to simultaneously control telephony, message switching and telex without fear of undesired interaction, and if desired, run background batch processing as well.

Acknowledgement

I would like to thank the many colleagues on whose work this paper is based and the Directors of the Plessey Company for permission to publish it.

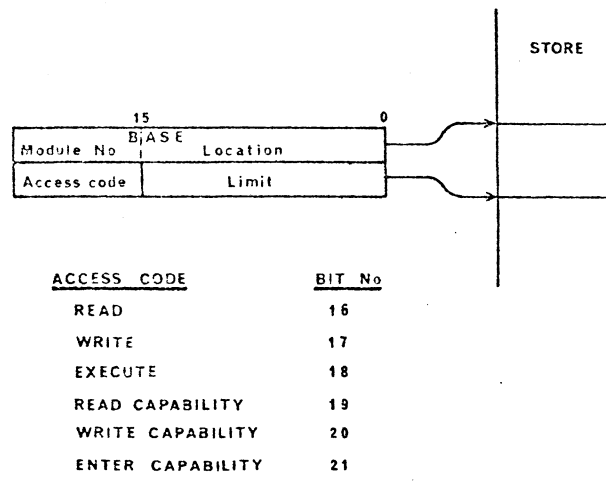


FIG. 2 CAPABILITY FORMAT

Reference 1) Dennis J.B. - Van Horn E.C.
(1966) Programming Semantics
For Multi Programmed Computations.

Comm. of the ACM, Volume 9
Page 143

HARDWARE OF THE SYSTEM 250 FOR COMMUNICATION CONTROL

D. Halton

Plessey Company Ltd.
Poole, Dorset, England

Abstract

The paper describes the implementation of a system designed to meet the requirements discussed in the associated papers. In order to meet the requirements of reliability and flexibility a multi-processor approach was adopted, allowing of modular and independent addition of processing and storage capacity. The main aims for the hardware design have been to achieve reliability utilising the principle of redundancy and to impose minimum restrictions on system and software organisation.

A multi-processor approach, while offering the ability to provide redundant modules in an economic manner, as opposed to full duplication, presents considerable problems in interaction between processors, particularly in a load-sharing mode. To deal with these problems and also to facilitate software organisation, a capability protection concept has been implemented in the system.

System Configuration

The control of the system and all processing functions are carried out by the Central Processors (CPU), the programs and data being held in the storage modules which are fully distributed i.e., all storage modules can be accessed by all processors. Thus, the functioning of the Central Processor is of major importance in an understanding of the system. Before dealing with the Central Processor in detail however, it is useful to consider the various modules of the system and the ways in which these can be configured. At this stage, the basic functions will be described with more detailed descriptions as appropriate at a later stage.

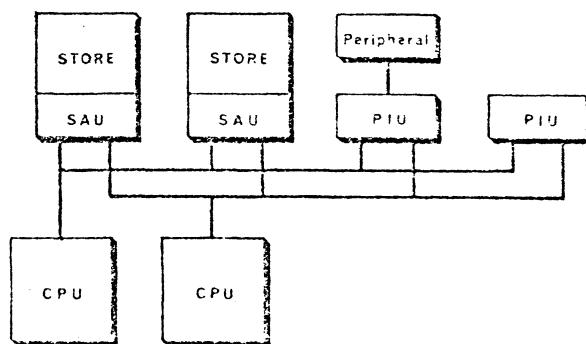


Fig. 1 Minimal System

Figure 1 shows a basic two processor configuration involving two storage modules and simple computer peripherals. The bus connection from the CPU to the Store Access Unit (SAU), known as the memory bus, is provided on a one per CPU basis, and is implemented in balanced drivers and receiver gates, switching about zero. Data is transferred in parallel between CPU's and stores. In this configuration the peripherals are connected directly to the memory buses, via Parallel Interface Units which comprise two sections. These are the bus interface, which is adopted as the system Standard Parallel Interface, and the section which converts the device interface to the Standard Parallel Interface. This method of peripheral connection is shown as an indication of system possibilities rather than a practical example.

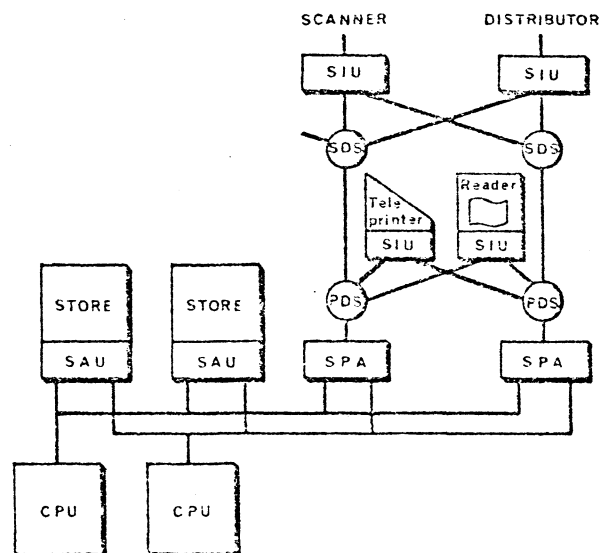


Fig. 2 Basic Exchange System

Figure 2 shows a rather more practical system which involves the basic connections to telephone peripherals. In System 250 the connection of the telephone system, which in computer terms involves a large number of slow-rate peripherals, is effected via a serial data medium, which also handles the slow-rate computer peripherals such as paper tape equipment and teletypes.

The interface between the parallel and the serial medium is effected by the Serial Parallel Adaptor (SPA) which handles data transfers in both directions. The serial medium comprises a number of data switches which can be cascaded to provide the required degree of concentration or diversion. Large numbers of physically distributed devices can be handled by situating a Primary Data Switch (PDS) adjacent to the SPA and Secondary Data Switches (SDS) adjacent to the devices. The PDS has a 64-way switching capability and the SDS a 16-way capability. The Serial Medium presents a Standard Serial Interface to all switching outlets and each dependent device must therefore have an interface unit which converts from the Standard Serial Interface to the device interface, known as the Serial Interface Unit. In some circumstances the demands of a peripheral device for data transfer may constitute a full load on a CPU so that the CPU is continually involved in control of peripherals. The inefficient use of the CPU is overcome in System 250, as in many conventional systems, by the introduction of a Channel Unit (CU). The Channel Unit is controlled by the system as another peripheral and is set up by the CPU to handle data transfers, thus enabling the CPU to release and carry out other processing tasks. The CU is provided with its own bus to enable transfers to be performed independently of the CPU's. The Channel Unit is thus an active system module, and the provision of multiple register sets enables it to control a number of simultaneous data transfer. The mechanisms which enable the CU to construct and use store addresses for the purposes of data transfer are subject to the same degree of fault detections as those in the CPU, which are to be explained in some detail.

Inherent in the input-output complex of System 250 is the concept of unified addressing whereby all data sources and destinations in the system are identified by a 24-bit address generated by a CPU. This means that in effect, all sources and destinations in the system are addressed as storage locations and this principle has two important corollaries.

- a) The CPU does not require explicit input/output instructions. A 'read data from peripheral' operation corresponds to a 'load data' instruction which specifies a peripheral register as the source of the transfer rather than a location in store.
- b) All transfers of data within the system are controlled by the capability protection mechanism. Thus the allocation and use of peripheral registers can be controlled by the supervisor program using the normal capability mechanism. This eliminates the need for a special executive mode of operation when handling peripherals.

The introduction of the Channel Unit as a further device, requiring its own bus, while improving the utilisation of CPU's, could increase the potential cost of interfacing peripherals onto the bus system. Every peripheral must be accessible to all active devices (CPU's and CU's) and so each PU would require a multi-port multiplexing function, where the number of ports equipped in a given configuration would be equal to the number of buses present.

Multiplexer module (BM), however, has been designed to concentrate the demands from the active modules onto a single Peripheral Bus, therefore removing the need for a variable port inlet facility on each PIU. The use of the multiplexer thus renders the peripheral device connections insensitive to growth in the form of additional active modules, since the extra buses simply plug into the multiplexer module.

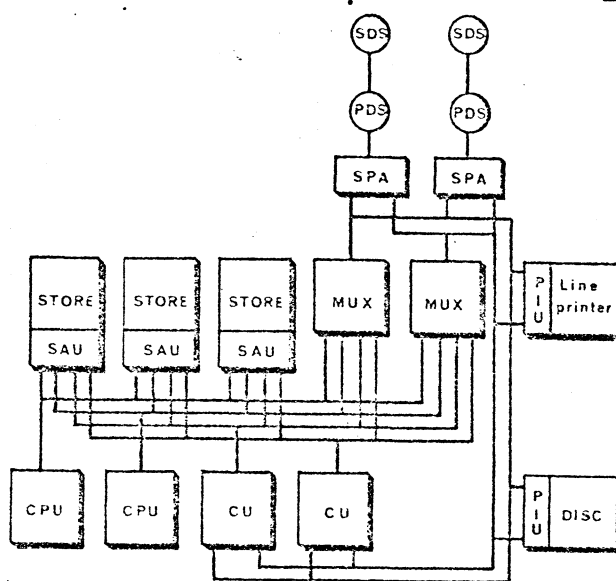


Fig. 3 Typical System Configuration

Fig. 3 shows a typical large system configuration involving Channel Unit and Multiplexers. High speed peripheral devices such as disks and drums are interfaced via PIU's to the peripheral bus, which utilises the Standard Parallel Interface; low speed devices and the telephone peripherals are connected via the Serial Medium and appropriate Serial Interface Units. Two multiplexers and hence two peripheral buses are provided to ensure system security in the event of a failure.

To complete the outline system description the storage module and the associated Access Unit should be mentioned. The Store Access Unit is a multi-port device, basically providing 4 ports and expandable in modules of 4 ports. The Store Access Unit (SAU) handles parity generation and checking on store transfers and deals with concurrent demands by processors on the same storage module on a queuing basis. The storage module initially provided is a 32K - 25 bit word plated wire store having a nominal cycle time of 250 nSec. In practice any store module can be utilised provided the interface conforms to the SAU specification.

The Central Processor Unit

In order to understand the principle of system operation it is necessary to describe the architecture and in particular the method of address construction of the Central Processor Unit. In a paper of this length it is not possible to cover all details of the processor, and the presentation therefore concentrates on the areas which are particular to System 250 and discusses

relatively briefly the aspects which conform to conventional computer technology.

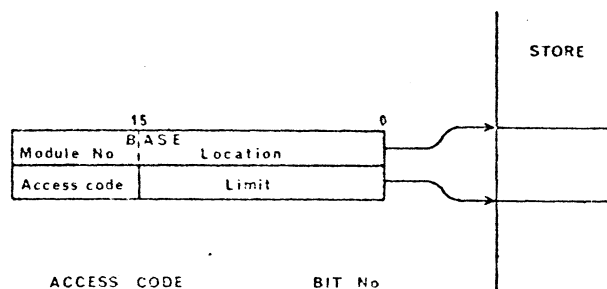
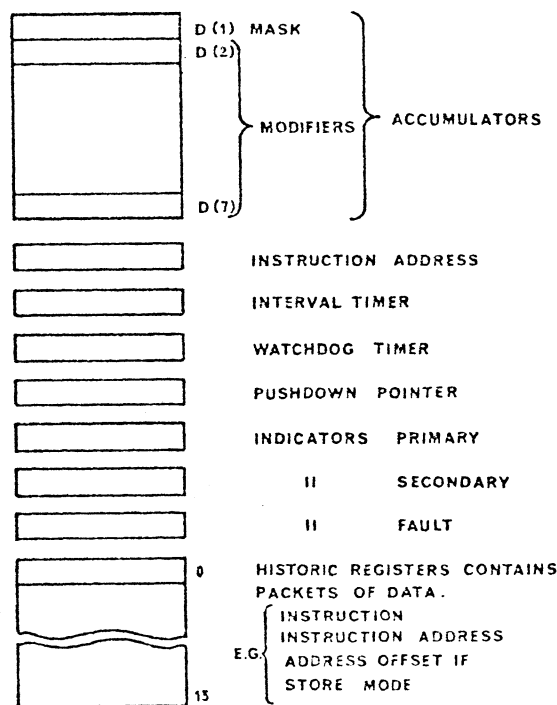
To introduce the principles of System 250 it is necessary to consider the concept of capabilities. Capabilities provide a method of addressing, from the central processor unit, to any other system module, in a defined manner which allows of the detection of a fault condition if the definition is violated.

The principle is most easily described in terms of access to main store, although as stated, capability protection is applied to all transfers in System 250. In the system, capabilities are loaded into hardware registers, the capability defining the lower and upper bounds of a block contained within a store module and the mode of access. All addresses are constructed using an address offset in the instruction, added to the base value contained in a capability register defined in the instruction.

The format of the capability register is shown in Figure 4. The register consists of two 25-bit registers, one bit in each register being parity. The access field contains an eight bit linear code which defines the operations permitted on the block defined by the base and limit. Three of the codes READ, WRITE and EXECUTIVE are concerned with operations on data and code, the other three are used for the manipulation of capabilities themselves.

The basic architecture of the machine is shown in Figure 5.

There are eight general data registers $D_0 - D_7$. All eight can be used as accumulators and $D_1 - D_7$ can be used as modifier registers. D_0 can also be used as a mask register which controls marked operation with certain instructions. Each register is 25 bits long, 24 bits + 1 parity bit being the basic word length of System 250.



ACCESS CODE	BIT No
READ	16
WRITE	17
EXECUTE	18
READ CAPABILITY	19
WRITE CAPABILITY	20
ENTER CAPABILITY	21

Fig. 4 Capability Format

There are eight general purpose capability registers, $C_0 - C_7$, general purpose in this context meaning that the registers are directly accessible to all programs. Each capability register comprises two 25-bit registers. As indicated in the diagram C_7 is allocated to holding the capability for the currently running code block, while C_6 defines the process capability pointer block which designates the capabilities which are available for the currently running code block. The remaining registers are loaded under programmer direction with capabilities for data blocks as required. By definition C_7 will always hold an EXECUTIVE access code.

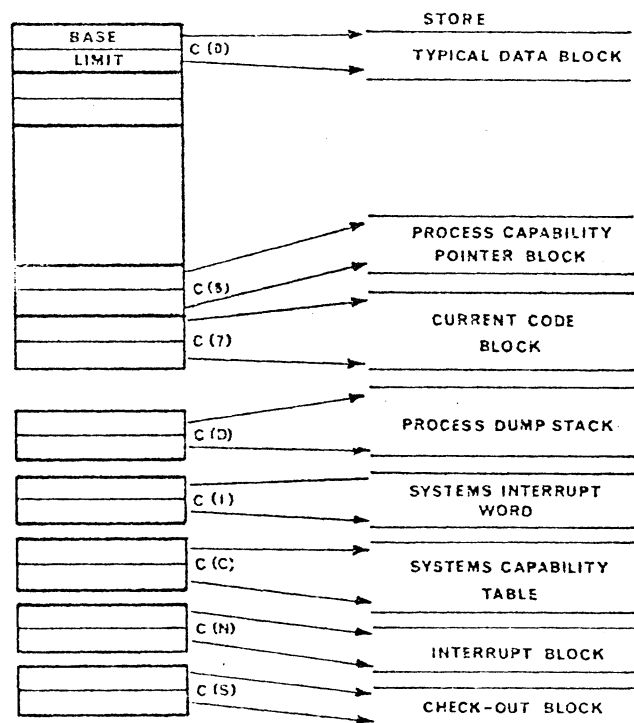


Fig. 5 Machine Architecture

In addition to the eight general purpose capability registers there are five special capability registers which are used by the processor and the operating system to access control information. These registers are not normally accessible to programmers, but can be accessed by programs which operate in a special internal mode. However, the hardware may allow the contents of a register to change during the execution of certain instructions. For reference the special capability registers are:

- C (D) - which defines an area of store called the Process Dump Stack.
- C (I) - which defines the System Interrupt Word.
- C (C) - which defines the System Capability Table.
- C (N) - which defines the normal interrupt block.
- C (S) - which defines the start-up block.

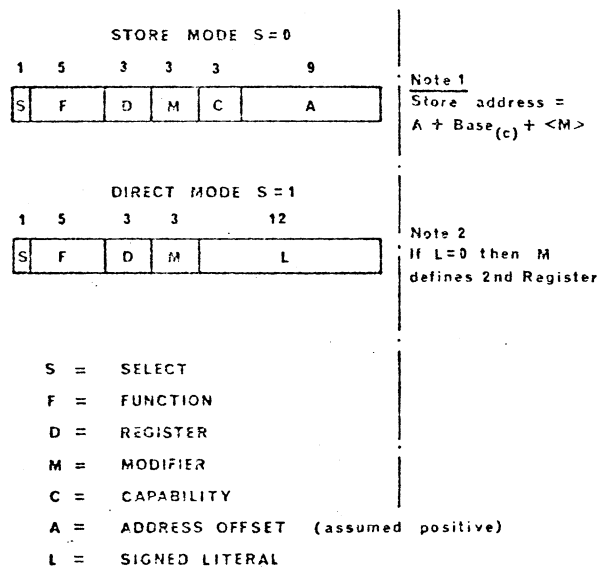


Fig. 6 Instruction Format

Instruction Format

The basic instruction formats of System 250 are shown in Fig. 6. There are two formats, known as Store Mode and Direct Mode, determined by the condition of the most significant bit, the S field.

In Store Mode, the address defined by the instruction is used to access a store location which contains the required operand. The F field defines one of 32 functions to be performed. The D field normally selects one of the general purpose data registers D_0 to D_7 to be used as the first register operand of the instruction.

There are two exceptions to this rule. Firstly, in the case of the JUMP instruction, the field is used to indicate one of eight arithmetic conditions. Secondly, two of the capability manipulation instructions use the field to define the capability register on which the operation is to be performed.

The M field selects one of the general data registers D_1 to D_7 to be used as a modifier in forming the address. Zero in this field indicates no modification and therefore D_0 cannot be as a modifier. The C field defines one of the general purpose capability registers C_0 to C_7 the contents of which define the relevant store block. The A field contains a positive binary number used in forming the operand address.

Address Construction

The address to be used by the instruction in Store Mode is formed by adding the Address Offset in the A field to the base value in the Capability Register defined by the C field. The resultant sum is then added to the contents of the modifier register defined by the M field, this step being omitted if the M field is zero.

Thus, an address is always defined relative to the base of a specified capability register, which in turn defines the store-block currently in use. It will thus be appreciated that the capability concept is an integral part of the processor operation. When the address is formed, checks are carried out to ensure that the constructed address is valid in terms of the capability definition. The Access Field is first checked for all zeros and if this condition is encountered, a PROGRAM trap is generated. The absolute address is then checked to ensure that the value lies between the base and limit values in the specified capability register. If either check fails a fault interrupt is generated. Finally the micro-program step which determines the form of access is checked to be compatible with the codes defined in the field. Again, if this check fails a fault interrupt is generated. If the checking procedure is successful the required store access is performed by the processor and the contents of the store location form the second operand. When the instruction address register is used as the address-source to access instructions, the same checks are carried out using the contents of capability register $C(7)$.

To appreciate how the capabilities are supplied, it is necessary to consider the 'enter' mechanism. 'Enter' is one of the available access code types, its purpose being to submit a subroutine call from one code to another, or more strictly a code block in one node to call a code block in another. The method of operation is as follows:

- 1) The calling node has an enter capability for the main capability block of the called node. An offset down this block gives an execute type capability for the required code block.
- 2) The call is achieved by a 'CALL' instruction, which specifies the enter type capability and offset. The effect of the call instruction is:
 - i) To load C_7 with the execute type capability for the required code block.
 - ii) To load C_6 with the enter type capability for the called node's capability block. A read capability access code is automatically supplied in C_6 , so that the called node can read its own capabilities.
- 3) The old values of C_6 and C_7 referring to the calling node, are automatically preserved in a stack defined by one of the special capability registers. These values are returned when the called node performs the instruction RETURN. It should be noted that the enter type capability does not violate security rules, a calling node cannot gain access to the data block of a called node by reading them, only the called block can utilise the relevant data blocks. Both calling and called nodes are mutually protected from each other.

Instructions

The instruction set of System 250 may be considered as being in two sections + data instructions and capability manipulation instructions. The data instructions are essentially conventional the second set being provided for the necessary manipulation of capabilities described in the paper on system operation.

System Interrupts

An important aspect of the processor, particularly associated with the handling of input/output transfers is the method of dealing with interrupt.

It is implicit in a traffic-sharing system that any processor is capable of performing any process. It is therefore follows that input/output and interrupt must be processor independent.

In Direct Mode the S bit = 1 and no store accesses are involved. Thus, the C field is not required and the C and A fields are combined to form an 11-bit signed literal (L). The literal value can be modified by the contents of the register defined in the M field. If the L field is zero, the M field is used to define one of the registers D₁ to D₇ as containing the second operand, giving register to register operation.

Capability Structure

Thus a capability register defines the permitted bounds of a store block and the operations upon it which are valid. It is in fact, an access right to store. Any attempt by a program to refer outside these permitted bounds or to perform illicit operations is detected by hardware and the program is interrupted into a fault handler. The capability register thus has two functions:-

- 1) To provide an addressing base for access to the store.
- 2) To limit the scope of a program and thus contain its potential for store corruption in the event of a fault.

It should be noted that the protection is equally valid against both hardware and software faults.

The special capability register C₆ already referred to, defines the process capability block, containing blocks that the code may want to operate on. Some of these would be data blocks, some might themselves be capability blocks, giving access rights to a whole new range of blocks. Thus a whole network can be constructed consisting of code and data blocks interconnected by capability blocks as shown in Fig. 8.

Each of the modes consists of a capability block with satellite code and data blocks, and further satellite capability blocks giving access to further data blocks etc., the whole forming a nodal data structure on which its code blocks can operate. A code block can operate on this structure provided two capabilities are supplied:

- 1) For the code block itself; in C₇
- 2) For the node's main capability block, in C₆ giving access to all blocks of the node.

This implies a departure from the conventional method of handling interrupts, in that the interrupt signal must produce a system reaction, which can be handled by any processor, rather than being connected to individual processors. Interrupts are therefore handled via the storage system.

All processors and input/output channels have access to a word in store known as the system interrupt word. Each processor references this word by means of the special capability register C(1). Every 100 μs (or any suitable period defined for a given system), the processor automatically stops at the end of the current instruction and carried out a check for interrupt sequence. This is illustrated in Fig. 7.

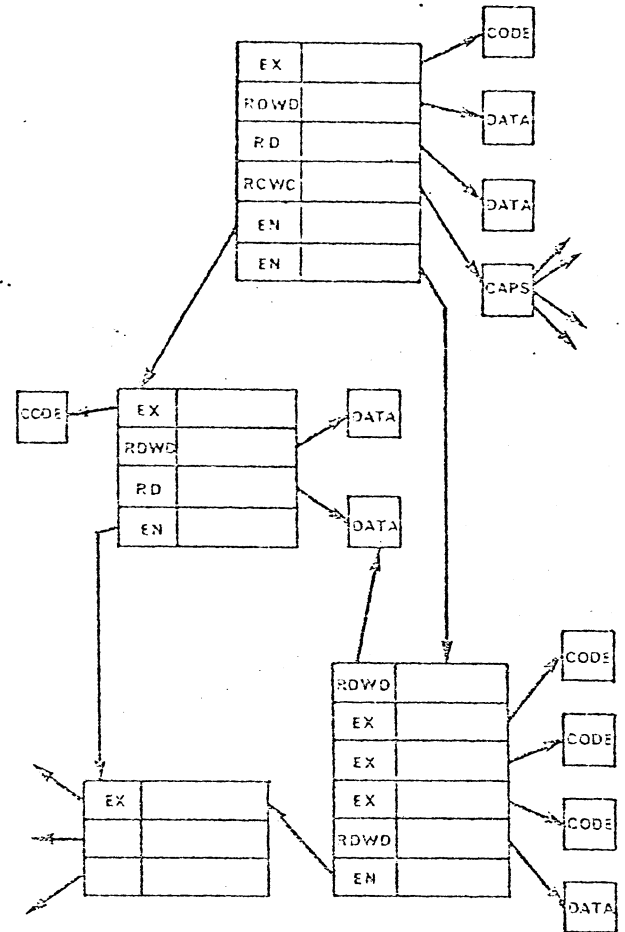


Fig. 8 Capability Network

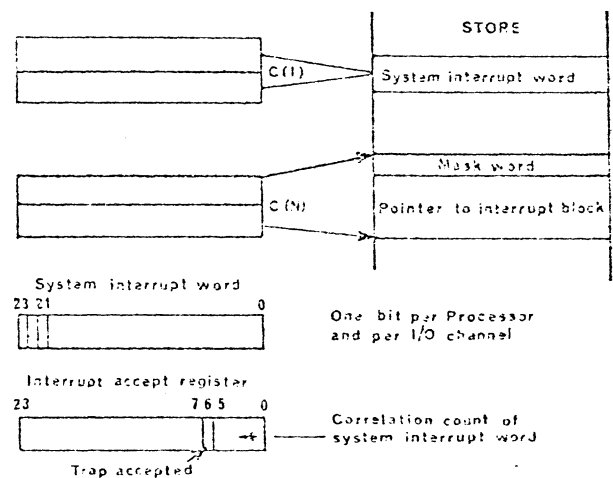


Fig. 7 Interrupt System

Using special capability register (CN) the processor fetches the Interrupt Mask Word. Using special capability register (CI), the processor fetches the System Interrupt Word (SIW) and locks the store module to prevent the processors from performing a simultaneous access. If the SIW is zero, the sequence is terminated. Otherwise, those bits of the SIW for which the corresponding bits in the Mask Word are one, are copied into an internal register. The value in the internal register is then correlated, i.e., a correlation count is produced which defines the position in the SIW of the first effective one, and this count is placed in bits 0 - 5 of the Interrupt Register, and the selected bit in the internal register is zeroed. The new contents of the internal register are then written back into the SIW location and the store is unlocked. The effect of this operation is to reset the most significant unmasked 1 in the System Interrupt Word and to define its position in the word by placing a correlation count in the Interrupt Register. This count is used by the processor to identify the module generating the interrupt, which may be a processor or an I/O Channel. In effect this means that the System 250 is not interrupt driven in the conventional sense. Input/Output operations are controlled by a polling action from the CPU's and interrupts are only used for such purposes as indicating the end of a drum or disc transfer, for example.

Protection Mechanism

In System 250, major reliance is placed on the capability mechanism to detect errors and to provide the basis for software structuring. It is therefore essential that the capability mechanism itself should be secure. Several protection devices are built into the processor to ensure this security.

Loading of capabilities is carried out from a system reference called the System Capability Table. A sum check word is held in the System Capability Table, which is the arithmetic sum of the base and limit values, right circulated by 9 bits. This is used as a check against store faults and corruption of the storage area containing the table.

The circulation of the sumcheck ensures that bit failures on the data paths into the processor will be detected as a sum-check failure. It should also be noted that any single bit failure in the store addressing mechanism will also generate a fault condition when the System Capability Table is accessed due to the three-word packet arrangement. Capability words are held in store and the registers as 24 bit words and parity. Parity is checked on loading and also whenever a capability register is accessed in normal usage. The circuitry of the capability registers, which are implemented in 16 bit MSI packages is arranged so that any single bit failure will generate a capability violation condition. The normal use of capabilities involves the arithmetic unit so that a valid base limit check confirms correct operation of the arithmetic unit. Parity checking is used on all transfers in the system. When an address is transferred to store, the processor begins to generate parity in parallel with this transfer. At the Store Access Unit, parity is also generated and returned on the parity check wire to the processor in parallel with the store access cycle. The parity condition which is odd parity over the 24 bits of the address is stored in both the processor and the Store Access Unit. When a data word is transferred to the store the processor again generates parity, over the 25 bits consisting of the data word and the stored address parity bit. A similar process occurs in the Store Access Unit, and the generated parity is returned to the processor over the parity check wire and written into the 25th bit of the addressed location.

The state of the parity check wire is compared at the processor with the parity generated locally and if a comparison failure is detected, the fault micro-sequence is entered immediately. The comparison occurs at Address Accept on both READ and WRITE operations and at Data Accept time on WRITE operations. This scheme ensures that parity generation and detection are kept to a minimum and that any single bit failure in the address selection logic in the store is detected.

A number of other detailed checks are included; to ensure for example, the continuity of the capability address highway, but space does not allow a full description.

To summarize, the capability mechanism is protected to ensure correct loading from the correct address to the hardware register, stability of the data during its life in the register and correct operation of the register whenever the capability is accessed.

A brief discussion of the actions which occur when any of the fault detection mechanism are activated is relevant at this point.

When any failure occurs the current process is immediately interrupted and RESET command sent to store. The processor then attempts to run a fault check sequence under the control of special capability register C_5 , which allows access to a special limited area of store containing the necessary parameters. A special check-out program is used to attempt to confirm the correct operation of the processor. If, during this sequence, a fault is encountered, the number of the module in special capability register C_5 is incremented by 1. Each store module has a special block per processor in corresponding locations, so that the parameters defined by C_5 are now loaded from a new store module and the process is re-attempted. Thus a processor which is in fact faulty can never return to system. On completion of a satisfactory check-out the action taken is system dependent. As a general indication, the processor will inform the supervisor and the supervisor will return the processor to system operation in a controlled manner.

Once the validity of the capability check is secured, the capability offers significant advantages in many areas of the system. Not least of these is the ability to constrain the functions of new modules, in both hardware and software, which are added to the system. The new modules can be tested with no risk of damaging the operation of the existing equipment. Thus the capability concept makes a significant contribution to the problem of expansion of a working system, the solution of which is vital in the telephone exchange environment.

Machine Construction

The Central Processor Unit and Store Access Unit are constructed using dual in-line IC packages mounted on 4 layer printed circuit boards measuring 190 x 153mm. The two outer layers carry circuit tracks and the two inner layers are ground and voltage planes respectively. Packages used are predominantly Texas 74H series, but other packages are used in some areas, for example, the capability registers which utilise 16 bit MSI scratch pad packages. Maximum package density per board is 42.

The boards plug into connectors which are mounted in printed circuit backplanes assembled in to a shelf. Each shelf can carry 33 boards. The CPU comprises two shelves, and the SAU one shelf. The backplanes themselves comprise a multilayer construction 12 layer in the case of the CPU and 10-layer in the case of the SAU. The store currently in use is a 32K 25 bit plated wire store having 250 ns cycle time and itself utilises similar construction techniques except that a full PC backplane is not

employed. Fig. 9 indicates the assembly. Fig. 10 shows a view of the equipment in a module environment incorporating special test sets.

Conclusions

While not exhaustive, it is hoped that this paper, in conjunction with the two previous papers, has indicated the principles of the System 250 approach, with its method of dealing with the problems of a multi-processor environment, which is considered to offer improved facilities over previously available hardware.

Acknowledgement

I would like to thank the many colleagues on whose work this paper is based and the Directors of the Plessey Company for permission to publish it.

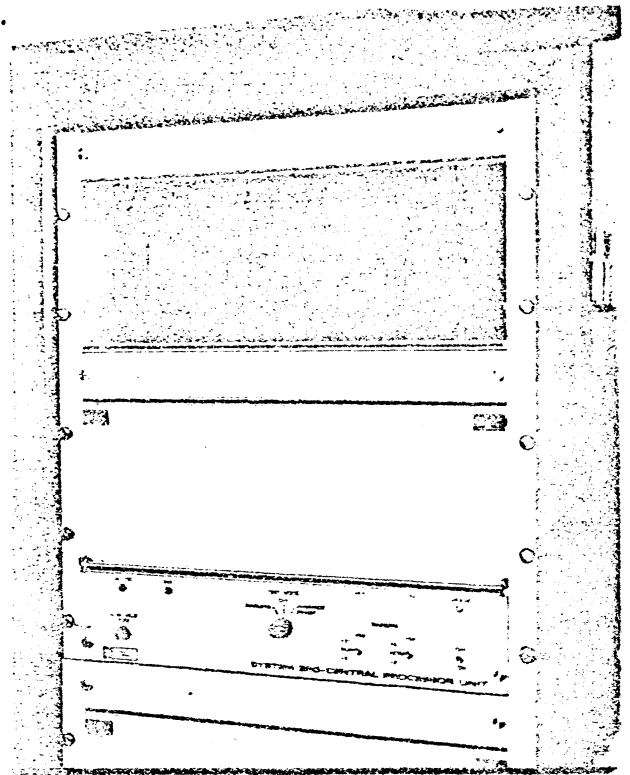


Fig. 9 Central Processor Unit of System 250

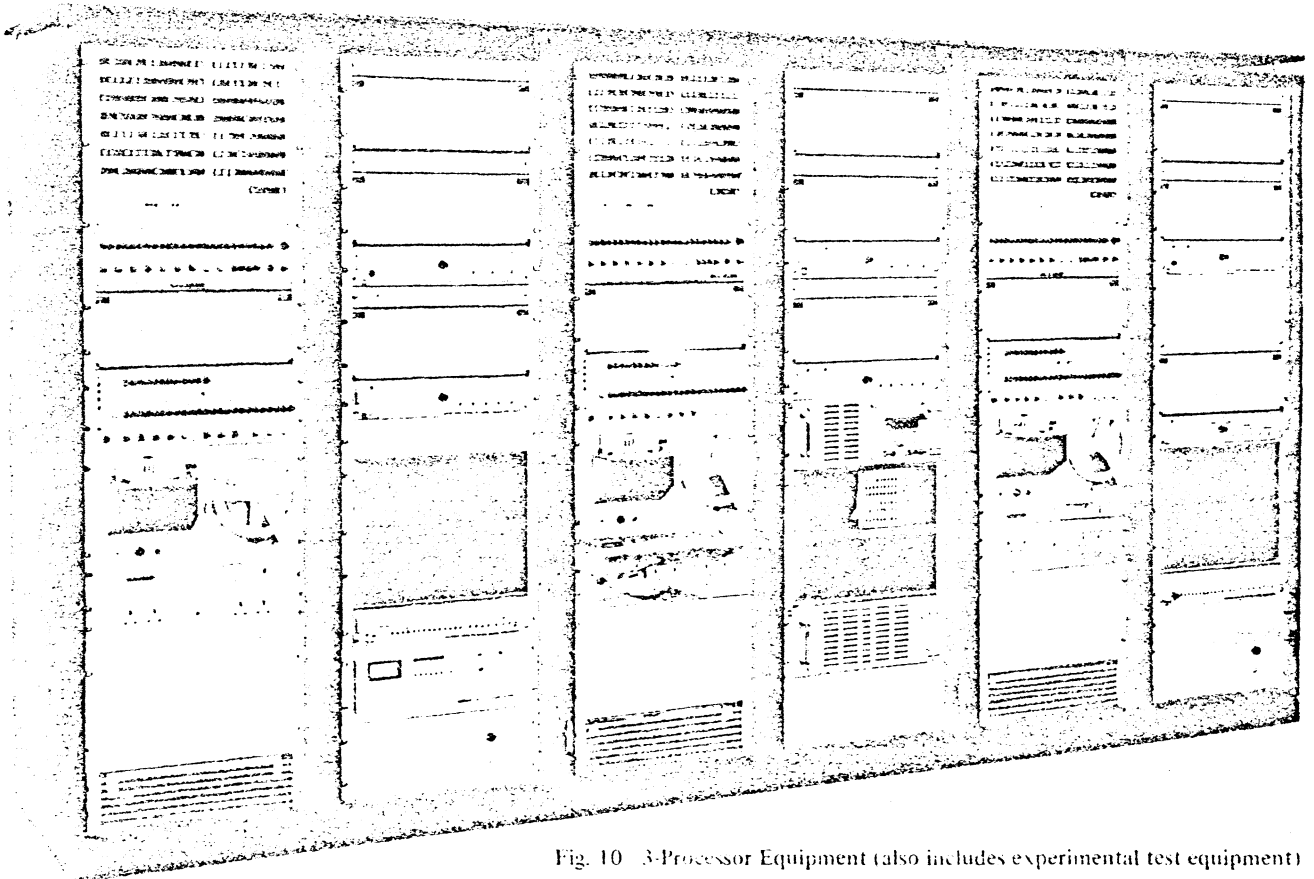


Fig. 10 3-Processor Equipment (also includes experimental test equipment)

TELEPHONE SWITCHING BASED ON SYSTEM 250

W.A.C. Hemmings

Plessey Company Limited,
Liverpool, England.

Abstract

Companion papers describe the requirements of a communications control system and give details of the hardware and software of System 250 for this application. This paper shows how System 250 is applied to telephone switching.

The telephone system so organised is modular and is capable of assembly to produce offices of various types and sizes. Whilst initially only autonomous offices may be needed, the system can grow to form networks based on the concept of a processing centre controlling a number of switching units, some or all of which may be remote from the processing centre.

1. The Overlay Concept

This paper is based on the 'Overlay' and Network Control concepts described in a companion paper and being considered for the continued development of networks such as that of the British Post Office (B.P.O.). These concepts cover all requirements from autonomous offices to full network control by Processing Centres.

It is proposed that the new network might be superimposed on the existing network, all new demands for service being met from the new network. Figure 1 shows the existing network indicated by the squares; the smaller squares are individual exchanges and they are shown linked star fashion to a Regional Office. The circles represent the new network, the smaller ones being remote units operated via data links to the Processing Centre. The main connection between the two networks is shown between the Regional Office (R.O.) and the Processing Centre. It is possible to have interconnections at a lower level than the R.O.; this would depend upon the local community of interest. Such an arrangement has several advantages over other methods of working and these are described in a companion paper.

1.(a) Implementation of the Overlay Concept

In order to implement the 'overlay' concept, we are developing a series of modular units from which the system is built, each of the modular units being themselves built from a number of standardised sub-systems.

Figure 2 shows a typical existing hierarchical structure comprising Regional Office, district offices, Local Offices etc. side by side with the new equipment. Three units are shown, the Main Switching Office (M.S.O.), Subscriber Switching Office (S.S.O.) and S.S.O. Concentrator. The MSO combines

the features of the District Office and the Manual Board together with transit facilities normally available at a District Office. The SSO is the equivalent of a District or Local Office and can operate in a similar manner. The SSO concentrator is really a remote switching unit based on a parent SSO.

These Switching Units are controlled by the processing centre PI, the various units being coupled to the processing centre by control data links. The units controlled by a single processing centre together with the processing centre itself can for convenience be called a control area.

The processing centres of all control areas are able to interwork by means of an inter-processing centre, data link system.

1.(b) Sub-systems

Figure 3 shows the component parts of the two S.S.O. shown in the previous diagram. These component parts are standard sub-systems and all switching units can be assembled from a relatively few types of the standard sub-systems. The following sub-systems are shown:

1. Subscribers Sub-System (SS) whose main function is to handle all subscriber activities such as calling line detection, ringing, supervision and so on.
2. Transit Sub-System (TS) which provides a low loss interconnection point for all the traffic offered to it.
3. Interface Sub-System (IFS) which provides a complete signalling and switching interface between junctions to and from the existing network.

These sub-systems are partly hardware and partly software; the sub-division is illustrated in Figure 4. The upper half of the diagram shows the hardware which performs the physical connections and functions. The lower half indicates the software part that resides in the control complex; also shown is a software activity named 'call control' which handles the software packages associated with each sub-system via standard sub-system control interfaces.

The hardware of each sub-system comprises many items which are interconnected, and each of these devices is controlled in software by a 'handler' package. Some devices such as markers or auxiliary circuits may be common to more than one sub-system. In these cases the complete function (hardware & software) is 'loaned' on a temporary basis to the sub-system requiring service. The allocation of auxiliary circuits is such that the sub-system does not know if the service it gets is exclusive or shared.

Figure 5 shows the Subscriber Sub-System divided into its component parts:-

- (i) Subscriber line circuits and handler (SLC) — Subscriber

- calling condition:
- (ii) Subscriber Concentrator Switch (SCS). This is a three stage reed relay space switch catering for a maximum of about 80 erlangs of bothway subscriber traffic. It is controlled by a marker and its marker handler both of which are in turn controlled by the subscriber concentrator switch handler, whose function is to operate on a software map which contains a record of the complete state of the switch. This map-in-software technique will be mentioned again later.
 - (iii) Supervisory relay sets (SUP) are provided at the interface to the Transit Sub-System; their function is to detect subscriber clear-down signals and to isolate the DC requirement of the subscribers' lines from the rest of the system. It also provides an access point for auxiliary circuits.

Auxiliary circuits are associated with supervisory relay sets via an Auxiliary Access Switch (AAS) which may be controlled by the same marker as the SCS; the marker in this case is controlled by the AAS handler using the map-in-software techniques already mentioned.

The auxiliary circuits required for the subscriber sub-system include the following:-

- (i) Digit receivers.
- (ii) Tone senders for Ring Tone, Busy Tone etc.
- (iii) Ringing current senders.
- (iv) Coin control devices for controlling pay station calls.
- (v) Transmission test tone sender/receiver for testing transmission paths prior to switching through.

All the above auxiliary circuits and their handlers are controlled by the S.S. control which in turn is controlled by call control. The Interface Sub-System, which is generally similar in concept to the Subscribers Sub-System, is designed to interwork with all types of junctions with which the new and existing network will be interconnected. Auxiliary circuits, such as Senders and Receivers, will depend upon the signalling conditions required.

The Transit Sub-System is, as mentioned earlier, simple in concept and comprises either a software controlled analogue switch or a software controlled digital switch, capable of interconnecting other sub-systems. Software control again uses the map-in-software technique. Different switch designs will be necessary to cater for the various situations where the transit sub-system will be employed within the network.

1.(c) Call set up through Sub-Systems

Figure 6 depicts a typical subscriber-to-subscriber call progressing through the sub-systems so far mentioned.

- a) SLC1 detects a calling condition.
- b) Path 1 is set via SUP1 and AAS to DR. DR returns dial tone and collects digits.
- c) If the called subscriber is not busy, path 2 is set from R/S1 via SUP1, TS, SUP2 to R/S2. A test tone is then sent in each direction in turn to prove the path.
- d) Path 3 is then set giving ring current to the called subscriber and ring tone to the calling subscriber.
- e) When answer is detected by RC, the speech path is completed through SUP1 and SUP2, all auxiliary devices being released progressively as their tasks are completed.
- f) SUP1 and SUP2 detect their respective subscribers clearing and terminate the call.

2. The Processing Complex and the Processor Utility

The processing complex is the Communication Control

System 250 described in companion papers. As far as the design of the telephone switching modules are concerned, the processing complex has been regarded as a 'utility' in which we need to consider only what it does, rather than how it does it. Such a 'processor utility' forms a secure and defined environment in which application processes may run.

2.(a) Peripheral Data Transfer 'Utility' (See Fig. 7)

The medium for the transfer of serial data is a two stage data switch. The primary stage, which is associated with the processor utility, has up to 64 ports with balanced line drivers for distribution within an exchange. The secondary switch is associated with the periphery and has a present maximum of 16 ports and unbalanced drivers. On outgoing messages each switching stage uses part of the address section of the message to select its output port. Similarly on incoming messages each stage appends its port address.

To check the messages outgoing from the processor utility an identifying check code is wired into each peripheral connected to the serial medium; this check code is transmitted with each message intended for the peripheral. Comparison of the received check code with the allocated code at each peripheral confirms that the message has arrived at the correct destination.

The check code is also added to incoming messages as a further check on the integrity of the data transfer arrangements; all paths between the processor utility and the peripherals are duplicated. The whole of the data transfer arrangements is regarded as a further 'utility', referred to as the Peripheral Data Transfer Utility.

2.(b) Scanners and Distributors

Beyond the secondary data switch, which is associated with the peripheral, are scanners and distributors for data collection and distribution. Two modes of reporting from the scanners are used: exception mode in which a report is made only when an input has changed, or response mode which reports on command. This method of working considerably reduces the number of reports, compared with a scanning technique in which the state of each input is reported on every scanning cycle. In order to simplify scanner hardware and to preserve a standard message format, it is arranged that when a message needs to be sent, the report will consist of a complete scan field and not an individual input. A standard design has been produced which caters for both modes of scanning with minimal hardware and software changes.

The basic function of the distributor is to convert serial data arriving in messages from control to a staticised parallel format to drive peripheral devices. Check codes are used to check the integrity of messages; an incorrect check code causes the distributor to ignore all further information.

Normally, security is achieved in both the scanners and distributors by including these functions in security blocks, that is sections of equipment within which a failure would have only a limited effect on system performance. However, in case of the scanners, duplication or N+1 security techniques can be used where advantageous, such as in the case of small offices.

2.(c) Line Circuit & Switchblock

The subscribers line circuit, not illustrated, has an electronic loop detector and uses a reed relay to isolate the loop detector so as to reduce crosstalk and give better testing facilities.

Figure 8 shows the subscribers' concentrator switch which consists of three linked stages A, B and C using reed relay

matrices. To cater for various subscriber calling rates, three methods of adjusting the concentration ratio are provided:-

- (i) Varying the multiple of A units on the A-B links.
- (ii) Varying the number of AB networks on the BC links.
- (iii) Sub-equipping the A units.

The present design represents an optimum between cross-point minimisation, ease of extension, production costs and maintainability and has been confirmed by computer simulation techniques.

2.(d) Marker

The switch network is controlled by a marker, a hardware device controlled by a software marker handler within the processor utility. The current state of the network is contained within a software 'map'. Information in the map is used to effect path choice and appropriate instructions are passed to the marker via the marker handler in order to set up the chosen path.

The marker operates in three modes

- 1. To set up paths.
- 2. To release paths.
- 3. To inspect and report on the busy/free condition of selected links.

To obtain adequate security the marker is duplicated. To prevent a switch network fault from disabling both markers, it is arranged that the switch network is divided into sections which can be isolated from the marker.

2.(e) Equipment Layout

Before concluding this brief review of the telephone hardware I would like to mention equipment layout and equipment practice. A new equipment practice has been developed for automated production having metric dimensions. Racks are single sided, are available in modular sizes allowing four heights, three widths and three depths, and have been designed either to stand as individual racks or be incorporated into suites of racks. Normal arrangements are made for providing telephone power supplies, fused AC outlets and other services, and blowers can be incorporated as required. Removable covers are provided at the rear of the racks.

Of the four heights of rack available, that proposed for telephone switching applications is 3198 mm high, being the metric equivalent of the current B.P.O. standard rack (10 ft. 6 in.).

Three modular standard rack widths are available, 600, 900 and 1150 mm. One of these, 1150 mm, will be used for all main analogue switching racks.

Shelf height can be chosen in 32.5 mm modular steps, the maximum height proposed being 285.5 mm. Unit width may vary from 25 mm to 150 mm in 25 mm modules. Typically, crosspoint relays and relay sets mounting comparatively large components such as relays and transformers require 75 mm units. Electronic units, i.e. Markers, Scanners, etc., are normally mounted on 25 mm or 50 mm units. Units of 75 mm or larger are available to mount special requirements such as power supply converters etc.

3. Software Structure for Telecommunications Control

3.(a) Application Software

Referring again to figure 4, each sub-system has a hardware part and a software part; the software handlers interface to an overall handler called 'call control'. The interface between the individual handlers and call control is standardised to enable each part of the application software to be separately dealt with as modules, and to allow modules to be added, deleted or

modified without affecting other modules.

The environment in which the application software runs is the processor utility which in any given circumstances will consist of a particular combination of many different pieces of hardware. The amount of equipment supplied is determined mainly by the traffic handling requirements of the processor utility; many of the individual tasks to be performed, however, are independent of both the particular configuration and the traffic load.

3.(b) Peripheral Handling

Peripherals are usually under control of a software peripheral handler package which within the system is responsible for establishing communication with the device, checking its status and sending all control signals necessary to perform the transfer. The Peripheral Handler is also able to deal with fault conditions in the peripheral and where appropriate attempt to repeat the transfer.

Not all peripherals come directly under the Peripheral Handler. Some peripherals are not shared and belong permanently to one application — for example a Marker. In this case operation of the peripheral is controlled by a package in the Application Suite, and only this package need have detailed knowledge of the characteristics and control of the device. All messages to and from the peripheral will be handled by the Peripheral Handler in order to ensure proper control and use of the data highways.

3.(c) Fault Security Routines

A fault monitor is provided which counts error reports from different packages and calls for File Audits. If the number of faults detected by the file audit routine becomes excessive then reference is made to a higher level routine which is part of the Operating System.

The fault security routines in the first instance initiate a reload of the applications read-only areas, and instruct the application to reconstruct or re-validate its read-write areas. The reload is performed from a second copy, maintained by the system.

Should errors still persist, a general system test and restart is initiated. A complete cycle of test programs is run, including such items as processor checkout, store tests and functional tests. Any faulty items should now be discovered and switched out of the system. The application is now instructed to revalidate or reconstruct the read-write areas again.

The next stage of error correction is system reload, when all programs and read-only areas of the system are reloaded and the lower level procedures are repeated. Finally, should any fault still persist, a trial reconfiguration is attempted, the various modules are then effectively switched out in turn until the error source is removed.

3.(d) Structure of the Application Program Suite

As mentioned previously each of the sub-systems comprise both hardware and software; the software functions are made up of one or more program packages, a package being regarded as a collection of blocks of code or data functionally independent and sharing a common Capability Pointer Table. The Capability mechanism is explained in a companion paper and will not be dealt with further here.

In a typical package structure, packages sub-divide into a number of function-oriented blocks:-

Normal Operation Code

Capability Pointers (which point indirectly to areas of store or other resources that need to be accessed)

Fault Messages
Package Restart Code
File Audit Code
Device Test Code
Private Files

The four code blocks all have separate entry and exit and each may call the Fault Monitor as a sub-routine.

3.(e) Fault Monitor

The fault monitor is responsible for all resources which the application has sole use of; the remainder are the responsibility of the 'System Fault Handler' within the Operating System. Program code, data, telephone hardware sub-systems are all in the Application category, whereas the stores holding the code and data, and the CPU's are in the System category.

Faults are detected by failure to complete a sequence, by data consistency checks, by built in hardware checks and by routineing. Faults are logged against resources and a fault count built up. The various resources are graded into a resource hierarchy in which Suite files are Primary resources; Secondary resources include Package code blocks, Tertiary resources include Auxiliary circuits, Supervisory relay sets and so on.

When the fault count on a primary resource exceeds a pre-set value, a request is made to the System Fault Handler to test the system and reconfigure if necessary. Overflow on secondary and tertiary resource fault counts are used to detect and put out of service faulty items, such as relay sets, which do not cause the entire system to fail. The fault count information is also used as a pointer to the application of further routineing and diagnostic processes.

3.(f) Interaction within the Application Suite

In an ideal situation the requirements of a subscriber making a call would be treated as a single dedicated transaction. This ideal situation cannot be met because of the volume of traffic to be handled and the slow rate at which data pertaining to a single call becomes available in relation to the speed of the system. Furthermore, the desired modularity of the telephone sub-system requires generally agreed interfaces which act as natural boundaries (sub-system functions communicate with Call Control via a standardised interface to allow a modular programming approach). This implies that in order to satisfy a service request separate processes are required in the sub-system function and the Call control function. Requests for service generally originate in a sub-system and so create a process in the sub-system suite. The latter is a collection of Peripheral Handlers co-ordinated by a sub-system control package; the sub-system functions, whilst being intimately familiar with the sub-system characteristics such as the trunking pattern, relay response time, equipment availability, etc., do not have any knowledge of the service requested. The Call Control function determines the service to be provided and in what manner, purely by requesting the sub-system suite to execute some standard functions. The process active in the sub-system suite will determine that a seizure has occurred and acknowledge this seizure to the Call Control.

3.(g) Map-in-Memory

I would now like to return to the subject of switchblock mapping and its integrity: our system uses a software map of the switchblock links. All path choices are made from this map, which means that path search and choice criteria, much too complex to realise in hardware, can be used. This leads to more efficient use of crosspoints, and junctions in the case of wide area control systems such as the 'overlay' concept.

In considering a software 'map' approach to path choice, care must be taken to prevent divergence between the state of the map and the state of the actual hardware. It can be expected that divergencies will occur: appropriate safeguards must therefore be built into the system to detect and eliminate any such occurrences. This is best done by the requirement that the normal traffic activity should provide a feed of information to the map which is sufficient to guarantee that the state of the map and that of the switchblock will converge. Such an arrangement will cover minor faults: by deliberate routineing activity on similar lines, rapid recovery can be made from even major fault conditions with the loss of very few of the calls concerned.

Our marker is arranged so that it inspects four switchblock inlets whenever it wishes to set or release a path which uses any of the four. The state of each inlet is checked against the map. If the marker should set up an incorrect path it may have used the correct switch inlet — if so the map will be correct again when the call clears. If it used the wrong inlet (and it was not trapped at this state because it picked one already busy), then two switch inlets in the map will be disagreeing with the hardware state. The busy inlet which is shown as free in the map will be freed when the call terminates. (This should not take long since this call can be expected to fail). The free inlet which is shown as busy in the map will be discovered when an adjacent inlet is selected for a subsequent call.

If the complete map should be lost due to store failure, a new map can be compiled from individual call records. These list the switchblock links used by all calls in progress. It would take several seconds to generate a new map.

An audit routine, based on constructing a new map and comparing it with the one in use, is used as a further check of map integrity; this only occurs on a small section of the map at a time in order to cause minimum disturbance, since calls could not be set up or cleared during the audit without excessively complicating the audit routines.

4. Acknowledgement

I should like to thank my colleagues upon whose work this paper is based and the Directors of the Plessey Company for permission to make this information available.

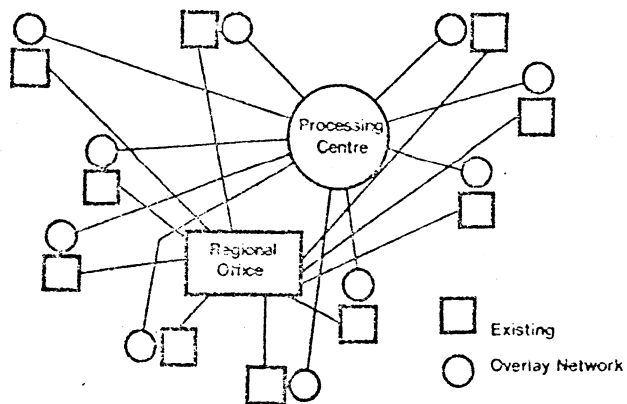


FIG. 1 'OVERLAY' NETWORK CONCEPT

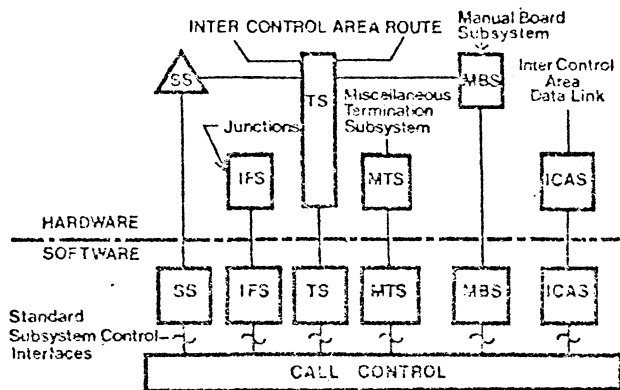


FIG. 4 SUB-SYSTEM HARDWARE & SOFTWARE DIVISION

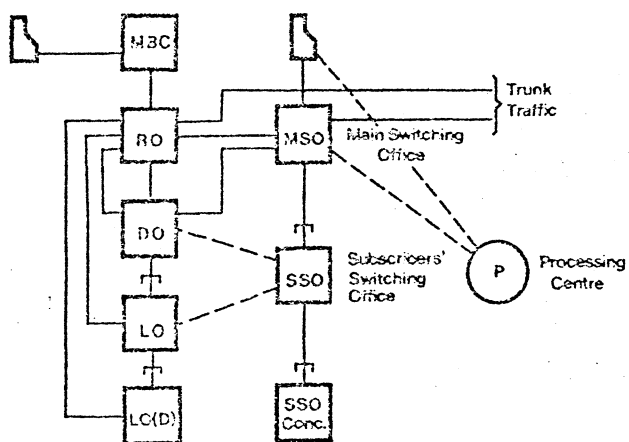


FIG. 2 EXISTING AND NEW CONFIGURATIONS

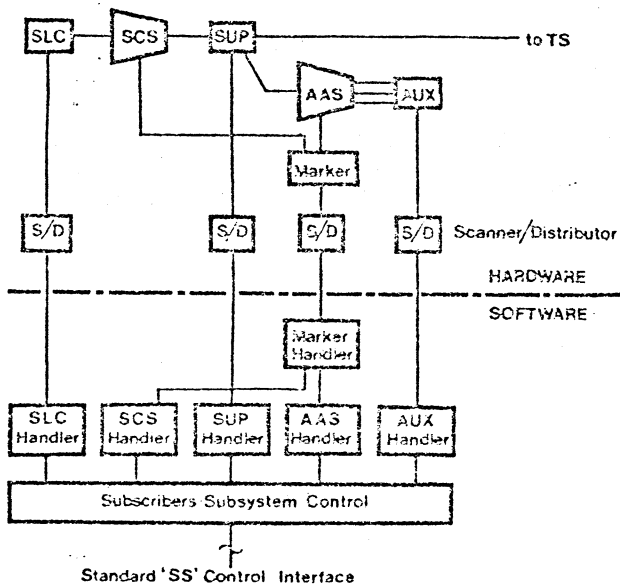


FIG. 5 SUBSCRIBER SUB-SYSTEM ELEMENTS

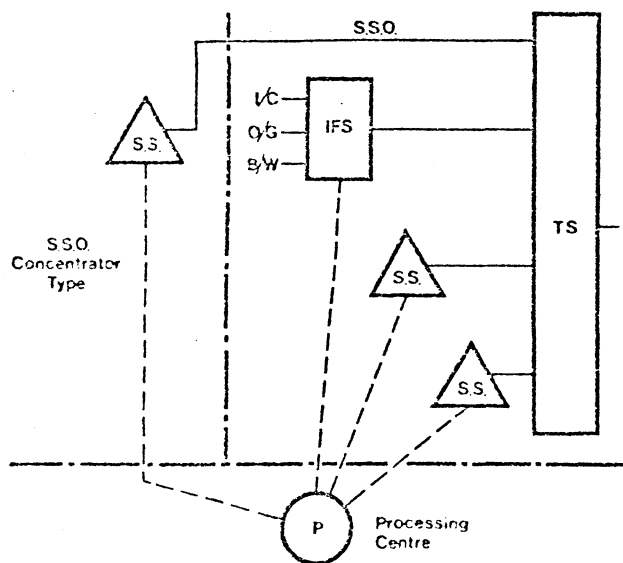


FIG. 3 SUBSCRIBER SWITCHING OFFICE, SUB-SYSTEMS

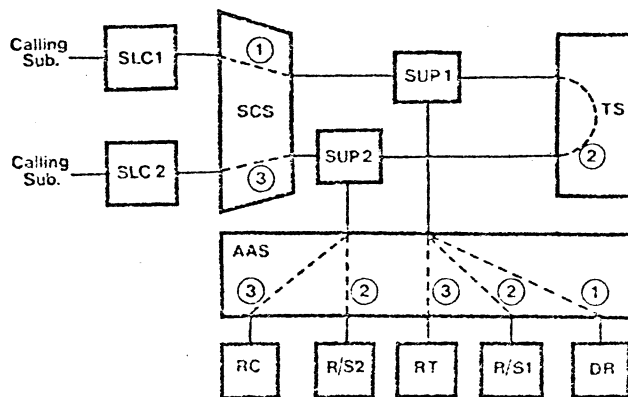


FIG. 6 SUB TO SUB CALL

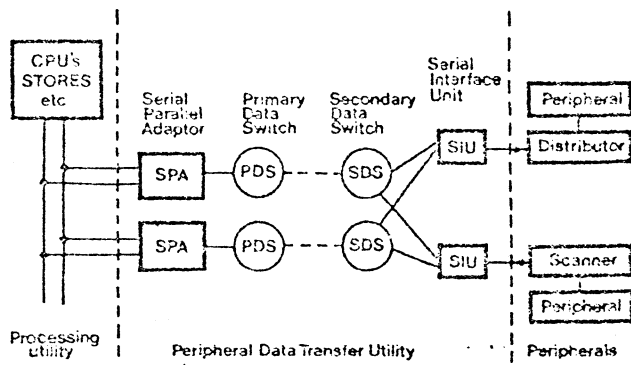


FIG. 7 PERIPHERAL DATA TRANSFER UTILITY

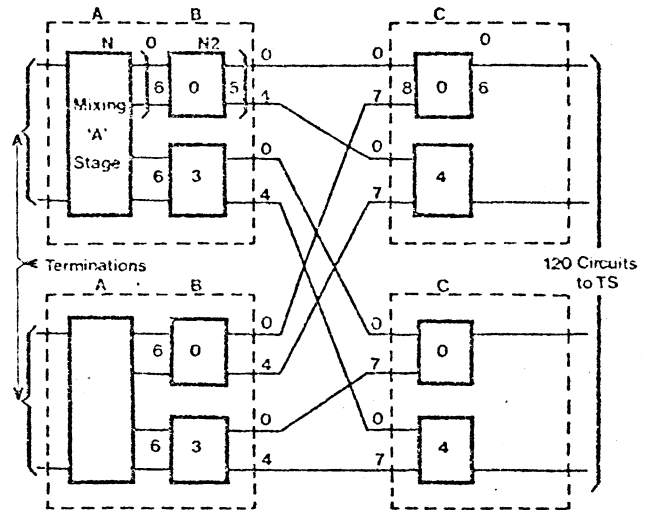


FIG. 8 SUBSCRIBERS CONCENTRATOR SWITCH